PROJECT-BASED LEARNING

PHASE-II EVALUATION

Project Title

TeamForge – Skill-Based Hackathon & Project Team Matchmaker

Team ID: DSCPP-III-2026-T238

Department of Computer Science & Engineering

Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team Name: TEAMFORGE

Team ID: DSCPP-III-2026-T238

Semester: III

Sections: ML2, ML1, DS2, ML3

Domain: DSA + OOP with C++

Team Members & Roles

1. Mukul Fartiyal – 2510370466
   Team Lead · Integration & web deployment
2. Anubhav Bisht – 2510370178
   Qt UI, testing & documentation
3. Siddhant Singh – 2510380233
   C++ / OOP design
4. Mehul – 2510370093
   DSA & matching

Mentor

Dr. Hriday Kumar Gupta

Interactions completed: _____

Roles follow the Phase-II report task allocation; each member demonstrates their own module.

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed a skill-based team matchmaker in C++/Qt. Topic approved on 31 Aug 2026; users, architecture, five-factor score and roles were fixed.

Feedback Received	Changes Incorporated
Availability (timetable) data is hard to collect reliably.	Availability replaced by **engagement** at the same 10% weight (1.0 if interested in the project, else 0.5).
A score alone does not explain why someone is suggested.	**Match insights** show the factor-by-factor and skill-by-skill breakdown.
The prototype should be easy to try and demonstrate.	The same app now runs **in the browser** (WebAssembly) with sign-in and shared data.

Five-factor score
weights total 100%

40%	25%	15%	10%	10%
Coverage	Complementarity	Experience	Engagement (new)	Diversity

Problem Statement & Finalized Objectives

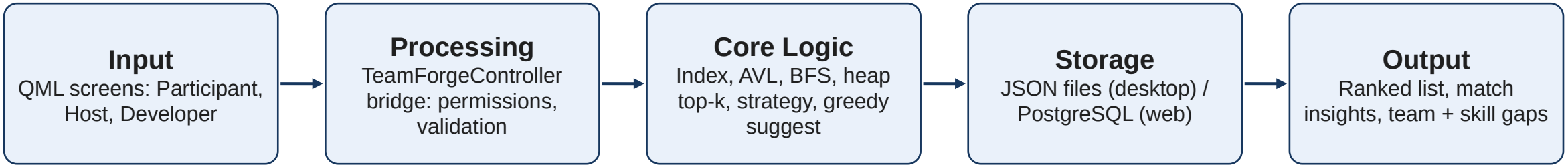
Problem Statement

Teams formed through friends often repeat skills and miss roles. TeamForge compares the skills a project needs with the skills students offer, ranks candidates with an explained compatibility score, and shows what the team still lacks.

Finalized Objectives

		Evidence in the current build
1	Participants create skill profiles (levels 1–5) and express interest in projects.	*Participant workspace; 86 sample interest requests*
2	Hosts get a ranked, explained shortlist from the 560-profile sample dataset.	*Match insights; 82.2 score checked by hand*
3	Build teams with live skill-coverage and gap checks, and save them.	*Team builder: duplicates, team size, missing skills*
4	Apply TCS-302 data structures and a TCS-307 OOP design in the matching engine.	*Developer workspace shows index, AVL height, BFS*
5	Offer the same application online with accounts and shared data.	*Live site; 52 server API checks pass*

Updated System Architecture / Workflow



Screens never compute scores: QML calls the bridge, the bridge calls the C++ core, and results come back as data.

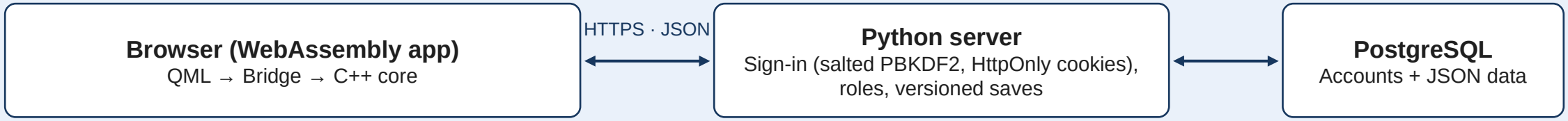
Major Components

- C++17 core (`src/core`): matching engine + data structures, no Qt types in headers
- `TeamForgeController` bridge to 51 QML screens
- Python server + PostgreSQL for the web version

Changes Since Phase-I

- Participant, Host and Developer workspaces
- Interest workflow: apply, review, accept / decline
- Browser build with versioned saves

Web deployment path



Module Design & Functional Requirements

Major Modules

1. Profiles & Projects

`Student, ProjectRequirement, Repository<T>`

Skill profiles (levels 1–5) and project requirements

2. Matching Engine

`SkillIndex → MatchingStrategy → rankTopK → suggestTeam`

Retrieve, score, rank and suggest

3. Team & Interest

`Team, InterestRequest`

Team validation; apply, review, accept / decline

4. Storage & Web

`JsonStorage, Python server`

Versioned JSON locally; PostgreSQL online

Functional Requirements

- Create and edit profiles with skill autocomplete (AVL prefix search)
- Rank candidates with the five-factor score
- Build, suggest and save teams with a gap check (duplicates, size, missing skills)
- Express interest; host reviews, accepts or declines
- Sign in online; shared data with conflict-safe versioned saves

Module Dependency



On every profile change the engine updates the hash index, AVL tree and skill graph through one update path, so they stay in sync.

Technology Stack & Development Methodology

Technology Stack

Programming	C++17, QML, Python
Storage	JSON files, PostgreSQL
Framework	Qt 6.12, Qt for WebAssembly
Tools	Qt Creator, CMake, Emscripten 5.0.5
Deployment	GitHub Actions, Docker, Render
Version control	Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation — **in progress**

Why this stack: Qt keeps the whole project in C++ and runs the same C++ and QML code on Windows and in the browser. Each module has its own Qt Test program (7 in total); GitHub Actions rebuilds the WebAssembly app on every change, and Render runs our Dockerfile with a Render Postgres database.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures with C (TCS-302)	Hash map: skill → students	`SkillIndex`
	AVL tree with rotations; prefix search for skill autocomplete	`SkillBST`
	Graph + BFS for related skills	`SkillGraph`
	Min-heap (priority queue) top-k ranking in O(n log k); sorting, searching	`rankTopK`
	Greedy team suggestion	`suggestTeam`
OOP with C++ (TCS-307)	Classes, encapsulation, validated constructors	`Student, ProjectRequirement, Team`
	Abstract class, virtual functions, polymorphism (weighted & coverage-only strategies)	`MatchingStrategy`
	Class templates	`Repository<T>`
	Operator overloading	`MatchResult`
	Exception hierarchy and input validation	`TeamForgeError`
	File I/O (versioned JSON); smart pointers	`JsonStorage`

Live proof: the Developer workspace shows the skill index, AVL tree height and BFS related skills on screen.

Development & Implementation Progress

Module	Status	Work Completed / Remaining
Domain model & validation	**Completed**	Student, Project, Team, Interest; exception hierarchy
Matching engine & DSA	**Completed**	Index, AVL, graph, heap ranking, two strategies, greedy suggestion
Qt UI (3 workspaces)	**Completed**	51 QML screens, 25 reusable components, Match insights, 3D entry screen
Storage & sample data	**Completed**	Versioned JSON, SHA-256 seed stamp, backups, fixed-seed generator
Interest workflow	**Completed**	Apply, review, accept / decline
Web version	**Completed**	WebAssembly, sign-in, PostgreSQL, Render
Testing & validation	**In Progress**	Server and web checks pass; full Qt Test run still to be recorded
Strategy study & final docs	**Pending**	Weighted vs coverage-only comparison; README; Phase-III report

6 of 8
modules completed (75% by count)

1
in progress

1
pending

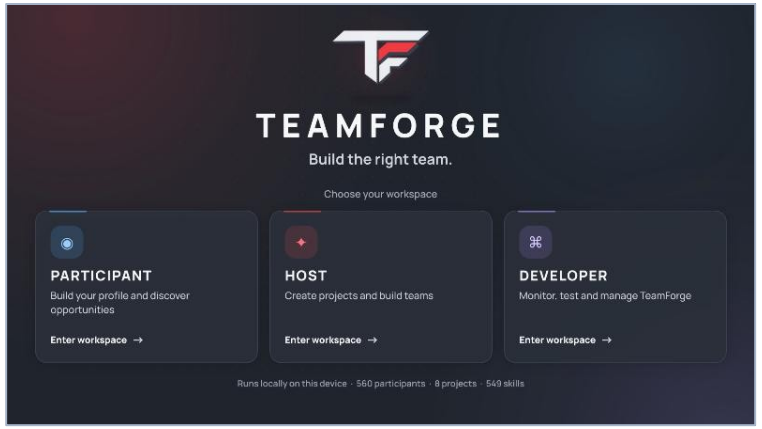
Sample dataset (fixed seed)
560 participants · 8 projects · 549 skills · 86 interest requests

Overall Project Completion is stated as a module count (6 of 8); it is not a separately measured completion score.

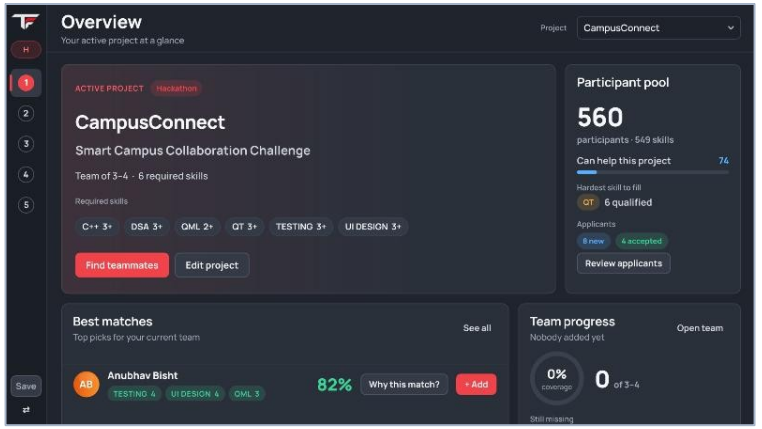
Working Prototype / Demonstration

Implemented Features

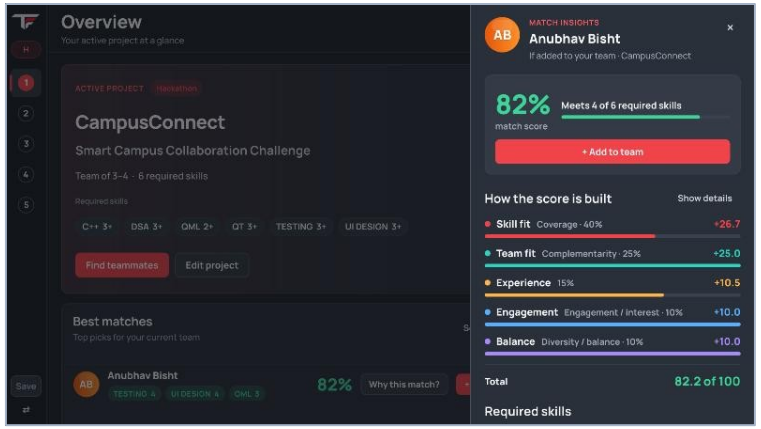
- Workspace entry with 3D logo
- Ranked best matches per project
- “Why this match?” factor breakdown
- Team builder, sign-in and shared data online



Entry screen



Host overview: best matches



Match insights: 82.2 / 100

Screenshots of the working application, captured from the live site **teamforge-geu.onrender.com**. Live demo during the presentation.

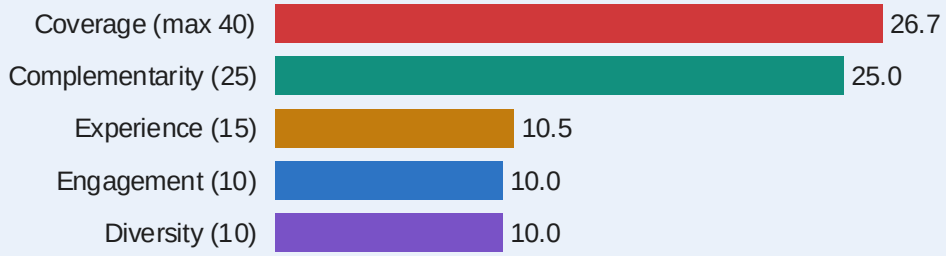
Demo flow

Enter Host workspace → Open CampusConnect → Find teammates → Why this match? → Add to team; check gaps

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Core compile (g++ -Wall -Wextra -Wpedantic)	No warnings	15 core files, 0 warnings	**Pass**
Score example (Anubhav, CampusConnect)	82.2 worked by hand	82.2 shown in Match insights	**Pass**
Qt Test / ctest suite (7 programs)	All programs pass	Full run not yet recorded	**To be recorded**
WebAssembly build (GitHub Actions)	Builds cleanly	Exit code 126 on first run; fixed, passes	**Pass**
Server API (52 checks)	Accounts, roles, conflicts	All 52 pass, incl. re-login keeps data	**Pass**
Page sync logic (11 checks)	Skip unchanged, flag conflicts	All 11 pass; changes saved	**Pass**
Live site (10 Oct 2026)	Healthy, sign-in, app opens	Health OK, sign-in, admin created	**Pass**
End-to-end use on live site	Testing checklist passes	Edit, reload, re-login being checked	**In progress**

Worked example: score breakdown (82.2 / 100)



Other observations

Ranking: O(n log k) over 560 profiles
App download: about 10 MB (gzip)
CI build 3–6 min · deploy about 35 s
Free-plan cold start: about 1 min

GitHub Repository & Development Activity

Repository Information

`github.com/anubhav8it/TeamForge`

Public; branch `main`
45 commits (as of 10 Oct 2026)

Contributors: `anubhav8it, mukulfartiyal8it, siddhantsingh013`
README.md: Phase-II section still says “Upcoming” (update pending)

Phase-2/ folder

`src/core`	C++ core
`src/app`	app + bridge
`src/web`	web bridge
`ui/`	QML screens
`tests/`	Qt Test
`data/`	sample data
`assets/`	images
`web/`	server, sign-in
`CMakeLists.txt`	build
`Dockerfile`	Render

Development Activity

Phase-I report and slides
Sep 2026

“Add files via upload”
Phase-II source

“Create / Update build-web.yml”
automatic WebAssembly build

“Create Dockerfile”
Render deployment

“Update CMakeLists.txt”
exception fix for browser build

“Build TeamForge for WebAssembly”
built files, by GitHub Actions

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts.**

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Mukul Fartiyal	Integration Lead	Controller and permission model, interest workflow, JSON storage, session and log, CMake build; WebAssembly build, sign-in server, PostgreSQL, GitHub Actions, Render; sample-data generator (with Mehul)	20
Anubhav Bisht	UI / Testing / Docs Lead	Qt Quick UI, 3 workspaces, Match insights, 25 reusable components, 3D entry screen; 7 Qt Test programs; Windows test build and testing checklist; documentation	21
Siddhant Singh	C++ / OOP Lead	Domain classes (`Student`, `ProjectRequirement`, `Team`, `InterestRequest`); `MatchingStrategy` hierarchy; `Repository<T>`; exceptions; input validation	1
Mehul	DSA Lead	`SkillIndex` hash map, AVL `SkillBST`, `SkillGraph` BFS, heap top-k ranking, greedy `suggestTeam`; sample-data generator (with Mukul)	0

Commit counts as of 10 Oct 2026: 20 + 21 + 1 + 0 = 42 member commits, plus 3 automatic GitHub Actions build commits = 45 total.

Individual Evaluation: every member will explain and demonstrate their own module live, from the code and the running app.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced & Solutions

Challenge	Solution / Learning	Status
Keeping index, AVL tree and graph in sync	One update path; rebuild functions and tests that compare rebuilt vs step-by-step state	**Fixed**
Old sample data loaded from a build folder	SHA-256 stamp on sample data; backup before replacing files	**Fixed**
First web version lost accounts on restart	Replaced with the real Qt app; data moved to PostgreSQL with versioned saves	**Fixed**
CI build failed (exit code 126)	Call CMake with Qt's toolchain file directly	**Fixed**
Browser app stopped on start	Qt kit disables exceptions; enabled -fexceptions for our code only	**Fixed**
Free hosting limits	Site sleeps after 15 min idle; database expires after 30 days	**Open**

Phase-III Roadmap

1. Run full ctest suite; record results
   Anubhav
2. End-to-end live-site testing; fix issues
   All members
3. Compare weighted vs coverage-only
   Mehul · on the 560-profile dataset
4. Move or upgrade the free database
   Mukul · before ≈ 9 Nov 2026
5. Update README (Phase II: “Upcoming”)
   Anubhav
6. Final demo, viva prep, Phase-III report
   All members

Expected Outcomes & References

Expected Final Outcomes

- Complete app on Windows and in the browser
- Explainable, reproducible candidate recommendations
- Team creation with validation and skill-gap checks
- Shared web access with sign-in and persistent data
- Measured weighted vs coverage-only comparison **(Phase-III, pending)**
- Usable in college hackathons, PBL and course projects

References

1. Cormen et al., *Introduction to Algorithms*, MIT Press.
2. R. Thareja, *Data Structures Using C*, Oxford Univ. Press.
3. E. Balagurusamy, *Object Oriented Programming with C++*.
4. Gamma et al., *Design Patterns*, Addison-Wesley.
5. The Qt Company, *Qt 6 Documentation; Qt for WebAssembly*.
6. cppreference.com; Emscripten and Render documentation.

Thank You

Questions & Suggestions